Research Progress

Paxos Simulator and RAG Research Direction

Chiharu Tomimori

2026/7/6

Presentation Overview

- Today, I will explain two parts of my recent progress.
- First, I will explain my educational Paxos simulator.
- I refactored the simulator, added a simple network layer, and implemented failure scenarios.
- Second, I will explain my current research direction after reviewing recent RAG papers.
- The main goal is to connect my implementation practice with my broader research interest in distributed systems.

Part 1: Educational Paxos Simulator

- I extended the simulator beyond the basic Paxos flow.
- The goal is not to build a production-level Paxos system.
- Instead, the goal is to understand why Paxos needs:
  - proposal numbers,
  - Prepare / Promise,
  - majority agreement,
  - careful message handling.
- I focused on making failure scenarios visible through code.

Paxos Simulator: Main Updates

- I made three main updates.

1. Refactored `simulator.py`
2. Added `network.py`
3. Implemented failure scenarios

Main Goal

Understand why Paxos is designed this way by observing how it behaves under failure scenarios.

Refactored Simulator

- Before refactoring, `PaxosSimulator` handled most of the logic by itself.
- It created Prepare messages, collected Promise messages, selected values, created AcceptRequest messages, and collected Accepted responses.
- This worked, but it made the roles unclear.
- After refactoring, I separated the logic into three parts:
  - `Proposer`: creates messages and stores responses.
  - `Acceptor`: decides whether to promise or accept.
  - `PaxosSimulator`: coordinates the overall protocol flow.

Why Refactoring Helps

- Paxos has clear roles in theory.
- If all logic is inside the simulator, it becomes hard to see what each node does.
- After refactoring, each class has a specific responsibility.
- This also makes future extensions easier.
- For example, when I added `network.py`, the simulator did not need to contain all network-related logic.

Key Idea

The simulator should control the flow, but the Paxos logic should belong to the nodes.

Network Layer

- I added `network.py` to simulate an unreliable network.
- It does not use real sockets or TCP.
- Instead, it manages messages inside the program.
- The main operations are:
  - `send`
  - `delay`
  - `drop`
  - `deliver_one`
- This allows me to explicitly control message delivery, delay, and loss.

Leader Conflict Scenario

- This scenario intentionally skips Prepare and Promise.
- P1 sends `AcceptRequest(1, A)` to A1 and A2.
- Since A1 and A2 form a majority, value A becomes chosen.
- Then P2 sends `AcceptRequest(2, B)` to A2 and A3.
- A2 and A3 also form a majority, so value B also becomes chosen.
- Now two different values are chosen for the same slot.

Problem

This is a safety violation.

Why Paxos Prevents Leader Conflict

- In real Paxos, P2 cannot simply skip Prepare and send AcceptRequest.
- Before sending AcceptRequest, P2 must collect Promise messages.
- If an Acceptor has already accepted a value, it includes that value in its Promise.
- In the leader conflict example, A2 would tell P2 that value A was already accepted.
- Therefore, P2 must inherit A and send `AcceptRequest(2, A)`.

Key Lesson

Prepare and Promise prevent a new leader from blindly overwriting a value that may already be chosen.

What I Learned from the Simulator

- Paxos safety depends on several mechanisms working together.
- Proposal numbers protect the system from old leaders and old messages.
- Majority agreement ensures that a value is chosen only when enough Acceptors accept it.
- Prepare and Promise allow a new Proposer to learn about previously accepted values.
- Refactoring helped me understand the roles more clearly.
- Network simulation made abstract failure cases much more concrete.

Part 2: Why I Am Interested in RAG

- I became interested in Retrieval-Augmented Generation, or RAG, through my previous university research.

Motivation

I want to study RAG from a distributed systems perspective.

Main Question

Main Research Question

How can we make RAG systems reliable when the knowledge base is dynamically updated?

- In real systems, documents are not static.
- Documents can be added, updated, or deleted.
- These changes must be reflected in chunks, embeddings, vector indexes, retrieved contexts, and citations.
- If the propagation is incomplete, the LLM may generate answers based on stale or deleted information.

Review Paper

Reviewed Paper

Security and Privacy in Retrieval-Augmented Generation: Architectures, Threats, Defenses, and Future Directions for Building Trustworthy Systems

- This review paper surveys security and privacy issues in RAG systems.
- It shows that RAG introduces risks not only in the LLM, but also in retrieval, indexing, context construction, and generation.
- The paper helped me understand that trustworthy RAG requires system-level design.
- This motivated me to focus on reliability issues inside the RAG pipeline.

Core Research Problem

Problem

A RAG answer may be generated from outdated or inconsistent evidence.

$$\text{Document} \rightarrow \text{Chunk} \rightarrow \text{Embedding} \rightarrow \text{Vector Index} \rightarrow \text{Context} \rightarrow \text{Answer}$$

- A document may be updated, but old chunks may remain.
- Embeddings may be generated from outdated text.
- Deleted documents may still exist in the vector index.
- Citations may point to old document versions.

Why This Is a Distributed Systems Problem

- Updates and deletions must propagate across multiple components.
- Different parts of the RAG pipeline may observe different versions of the knowledge base.

Research Direction

I want to study consistency guarantees for dynamic RAG systems, especially freshness, deletion guarantees, and snapshot consistency.

Overall Conclusion

- Through the Paxos simulator, I learned how distributed protocols preserve safety under message delay and leader conflict.
- Through the RAG paper review, I found that RAG also has system-level reliability problems.
- I want to study RAG as a distributed data system where versioning, freshness, deletion, and consistency must be guaranteed.